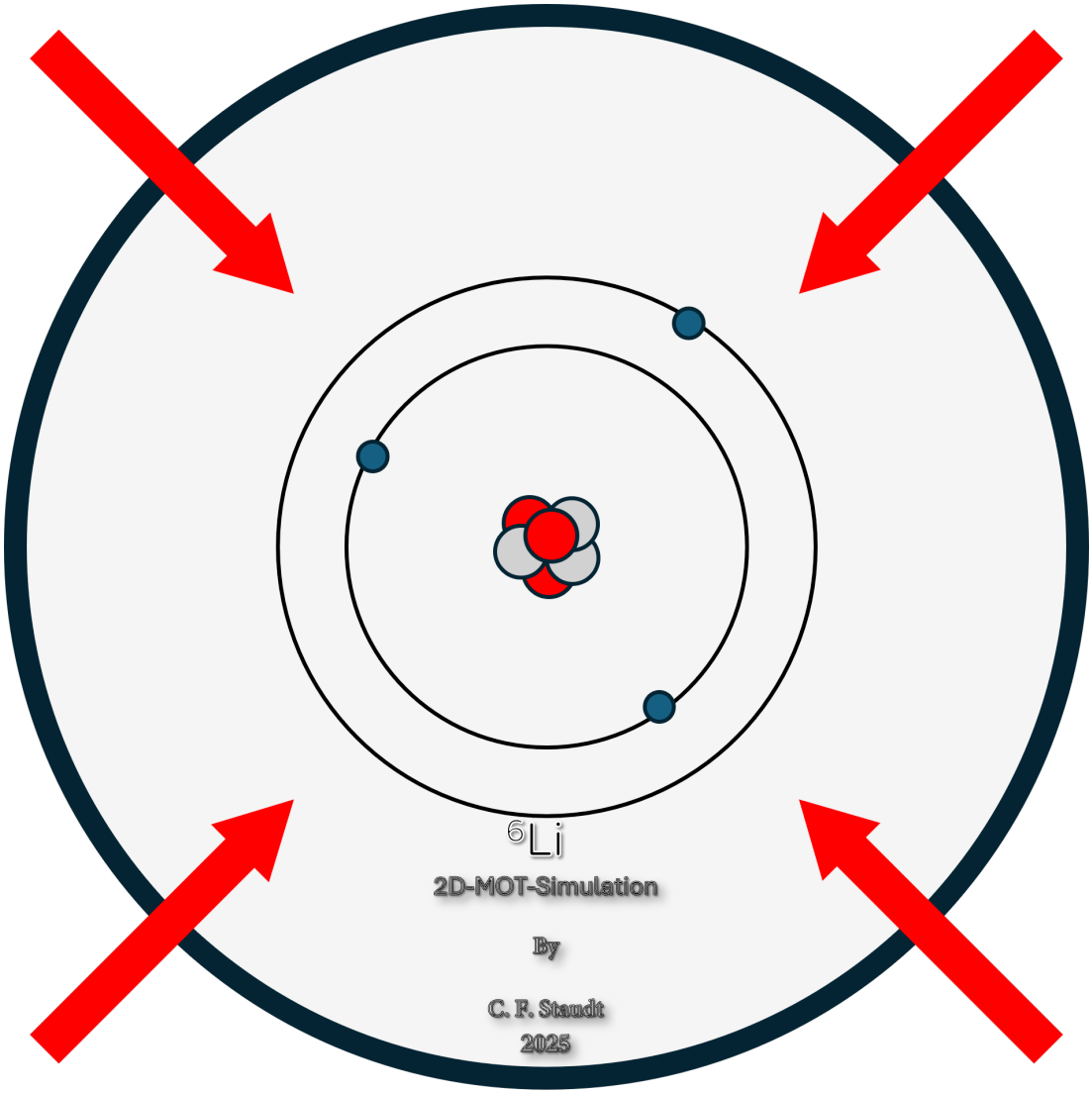
User Manual

2D MOT Simulation for Lithium-6



August 6, 2026

Contents

Introduction

What the simulation does

This software is a Monte-Carlo simulation of Lithium-6 atom trajectories through a two-dimensional magneto-optical trap (2D MOT) and push-beam region. It models, per atom, the absorption and spontaneous emission of laser photons, the Doppler and Zeeman shifts that set the scattering rate, gravity, and configurable laser and magnetic-field geometries. The physics core is compiled with Numba for speed.

Deep dive: what "Monte-Carlo" means here

Photon scattering is a random process: whether an atom absorbs a photon in a given interval, and the direction of the subsequent spontaneous emission, are drawn from probability distributions rather than computed as a smooth force. The simulation follows each atom individually, rolling these dice step by step. Running many atoms and averaging recovers the physical behaviour of the ensemble, together with its statistical spread.

It is a research tool: its purpose is to produce physically trustworthy Li-6 trajectories that can be compared against experimental data — atom distributions, push-beam velocities, and spectra.

At present only Lithium-6 is fully supported, but the tooling is written so that it can be extended to other atomic species.

Work in progress

Atom–atom interactions (collisions) are a work in progress and are not yet included in the simulation.

What the GUI provides

The GUI is the front end to this engine. From it you can:

- build and edit JSON parameter files, validated against a schema;
- run single or batch simulations and monitor their progress;
- generate Maxwell–Boltzmann oven / initial-condition samples;
- create parameter sweeps across many files (the Incrementor);
- compute frequency-scan spectra;
- generate interaction lookup tables (the Diagonalizer).

The same simulations can be run headless from the command line; both run modes are described in this manual.

About this manual

This **User Manual** (Part I) explains how to operate the GUI, tab by tab, and how to run simulations. The companion **Technical Manual** (Part II) documents the internal architecture and how to extend it with new models. If you are new here, start with chapter I.1, the Quick Start. This edition documents version 1.1.0 of the simulation.

Callout boxes used in this manual

Coloured boxes flag information that stands apart from the main text. The colour tells you what kind of box it is:

	Box	Meaning
	Note	General information or a useful tip.
	Deep dive	Optional "how does it work" background — the physics or mechanics behind a control. Safe to skip.
	Work in progress	Feature is present but under active development and may change.
	Known issue	Feature works but has a documented bug or limitation.
	Not implemented	Feature is a stub or absent; the box says what to use instead.

Chapter I.1

Quick Start

This chapter takes you from a fresh clone to a running simulation in a few minutes. Later chapters explain each screen in detail.

I.1.1 Prerequisites

- Python 3.12 (required).
- A virtual environment is strongly recommended.

I.1.2 Installation

From the repository root:

```
python -m pip install -r requirements.txt
```

I.1.3 Launching the GUI

```
python -m main --GUI
python -m main --GUI --style dark # dark theme
```

On startup the splash screen appears, then the main window opens on the **Simulation Cockpit** tab (Figure I.1.1).

Note

The first simulation you run triggers Numba JIT compilation, which takes 1–3 minutes. Subsequent runs use the cache and start immediately.

I.1.4 Running your first simulation

1. Click the **open-folder** button in the toolbar (or *File* → *Open Directory*) and choose the `setup parameters/` folder.
2. The JSON setups in that folder appear in the **Loaded File** list on the left. Select one, e.g. `Hammel_Setup.json`.
3. Its parameters load into the settings tree on the right. Review or edit them if you wish (see chapters I.4 onwards).
4. Press the **Run** button (▶) in the toolbar. Progress is shown in the status bar; log output appears in the bottom pane.
5. Use **Pause**, **Resume**, and **Cancel** in the toolbar to control a running simulation.

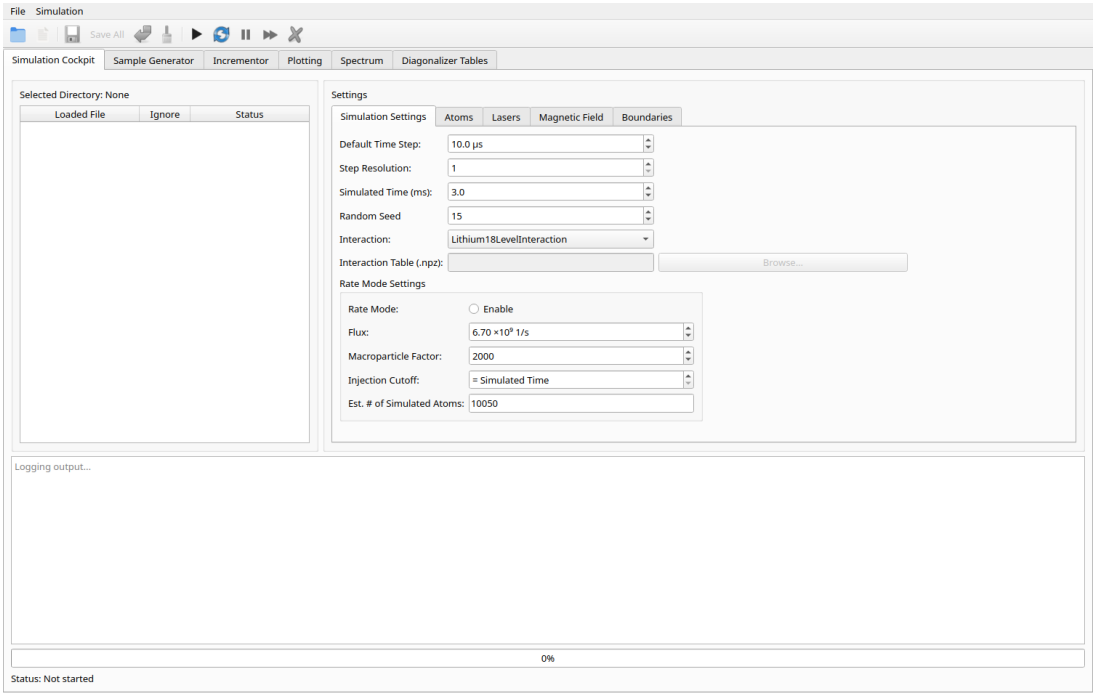


Figure I.1.1: The main window on startup, showing the six top-level tabs, the settings tree, the file list, and the log pane.

I.1.5 Where the results go

Each run writes to:

```
simulation_results/DD_MM_YY_N/run_N/
  result.csv # per-step, per-atom state
  config.json # copy of the parameters used
```

See chapter I.15 for the exact file format.

I.1.6 Running from the command line

For batch runs without the GUI:

```
python -m main --files "setup parameters/Hammel_Setup.json" --target-dir my_results
```

See chapter I.14 for all CLI options.

Chapter I.2

The GUI at a Glance

The main window has four regions (Figure I.2.1): the **menu bar**, the **toolbar**, the **tab bar** with the six top-level tabs, and the **status bar** at the bottom.

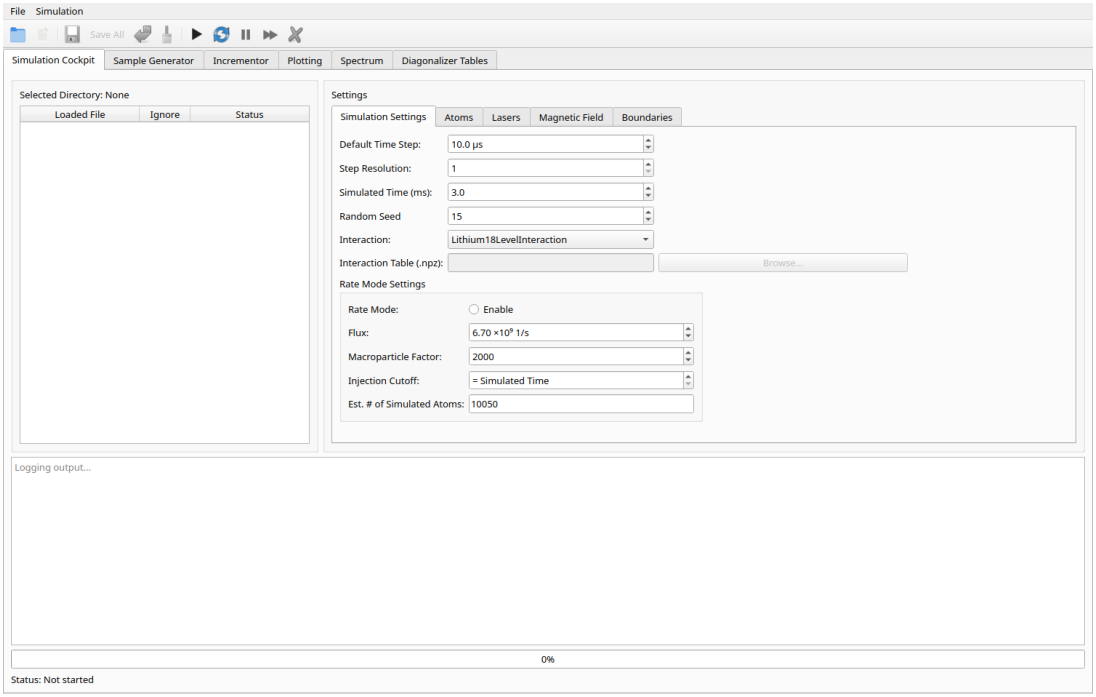


Figure I.2.1: The main window. Top to bottom: menu bar, toolbar, the six-tab bar, the active tab (Simulation Cockpit), and the status bar.

I.2.1 The six tabs

Simulation Cockpit Load parameter files, edit them in the settings tree, and run/monitor simulations. See chapter I.3.
Sample Generator Generate oven / initial-condition samples. See chapter I.9.
Incrementor Sweep a parameter across many generated files. See chapter I.10.
Plotting See chapter I.11.
Spectrum Compute frequency-scan spectra. See chapter I.12.
Diagonalizer Tables Generate interaction lookup tables. See chapter I.13.

Not implemented

The **Plotting** tab is currently an empty placeholder ("Plotting Tab – Shell"); no plotting functionality is wired up yet.

I.2.2 Toolbar

	Action	Shortcut	Purpose
	Open Directory	`Ctrl+L`	Load all JSON setups from a folder
	New	`Ctrl+N`	Create a new parameter file
	Save	`Ctrl+S`	Save the current file
	Save All	`Ctrl+Shift+S`	Save all modified files
	Discard Changes	`Ctrl+D`	Revert the current file
	Discard All Changes	`Ctrl+Shift+D`	Revert all files
	Run	`Ctrl+R`	Start the simulation
	Resume run	—	Continue an *interrupted* run from its checkpoint
	Pause	`Ctrl+P`	Pause the running simulation
	Unpause	`Ctrl+Shift+R`	Resume a *paused* simulation
	Cancel	`Ctrl+Shift+C`	Cancel the current operation

Note

Unpause resumes a run you paused in this session; **Resume run** picks up a previously interrupted run from its latest checkpoint. Save, Discard, Pause, Unpause and Cancel are disabled until they are applicable (e.g. Save activates once a file has unsaved changes).

I.2.3 Menu bar

File *Load*, *Save*, *Exit*.

Simulation *Run*, *Resume run*, *Pause*, *Unpause*, *Cancel* — the same simulation controls as the toolbar.

Chapter I.3

The Simulation Cockpit

The Simulation Cockpit is the central tab: you load parameter files here, edit them, and run and monitor simulations. It has three regions (Figure I.3.1): the **file list** on the left, the **settings tree** on the right, and the **log/progress** area along the bottom.

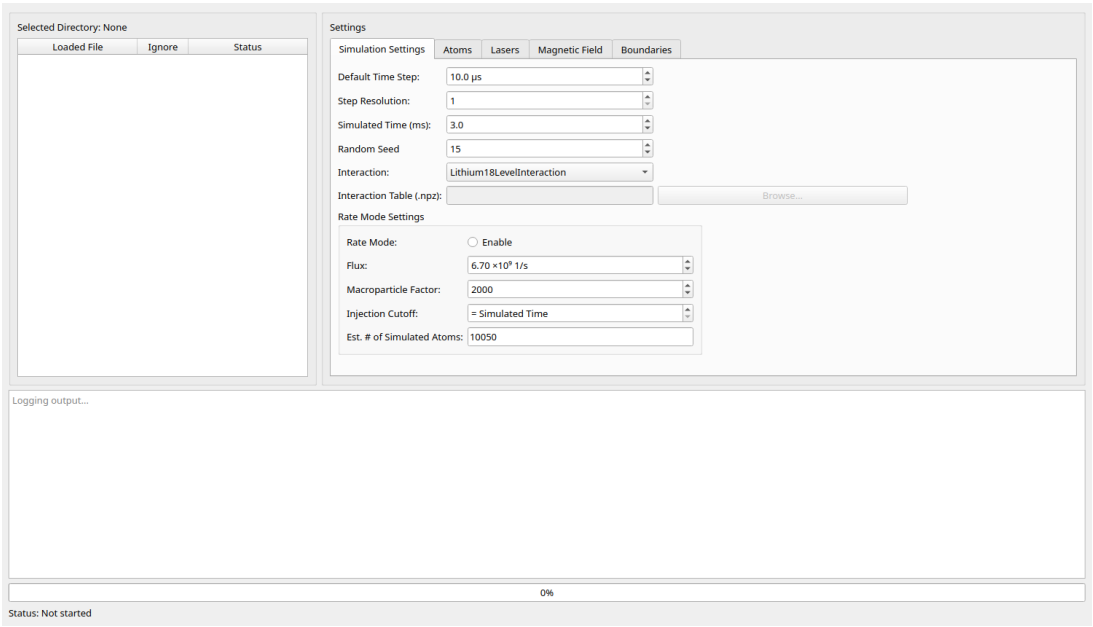


Figure I.3.1: The Simulation Cockpit: file list (left), settings tree (right), and log, progress bar and status line (bottom).

I.3.1 Loading parameter files

Click **Open Directory** (toolbar or *File* → *Load*) and pick a folder of JSON setups. Every `*.json` in that folder is listed in the file table, and the selected path is shown next to *Selected Directory*. Selecting a file loads its parameters into the settings tree on the right, and the heading changes to *Settings for:* followed by the file name.

I.3.2 The file table

The table has three columns:
Loaded File the file name.
Ignore tick to exclude a file from a batch run without deleting it.

Status the per-file state, colour-coded: before a run it shows validation (*valid* or *N error(s)*) and an *unsaved* marker for edited files; during a batch it shows the run state — *pending*, *simulating* or *done*.

From the table you can also **rename**, **copy** and **delete** files, drag rows to **reorder** them, and open a **validation/diff** view for a file. **New** (toolbar) creates a fresh file pre-filled from the JSON schema defaults.

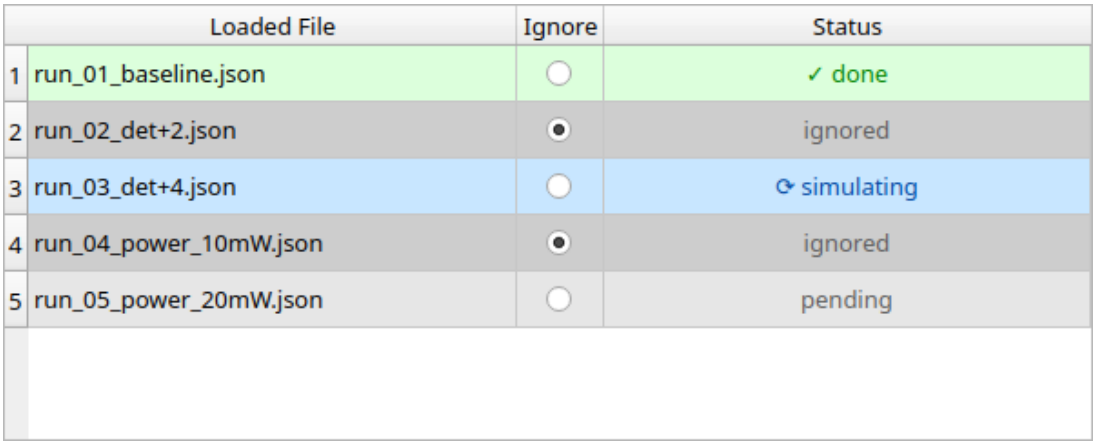


Figure I.3.2: A batch queue mid-run. The queue runs top-to-bottom: `run_01` has finished (done), `run_03` is currently simulating and `run_05` is still pending, while `run_02` and `run_04` are ticked *Ignore* and skipped over.

Note

Pressing **Run** simulates every *non-ignored* file as a batch, in the order the rows appear (top to bottom); ticked files are passed over without being deleted. Because the order matters, you can drag rows to rearrange the queue before starting it (Figure I.3.2).

I.3.3 Editing parameters

The settings tree groups all parameters into five sub-tabs — Simulation Settings, Atoms, Lasers, Magnetic Field and Boundaries — documented in chapters I.4–I.8. Edited files are tracked as "dirty"; use **Save**/**Save All** to write changes or **Discard**/**Discard All** to revert.

Note

Edited files are flagged *unsaved* in the Status column. If you start a run while any file still has unsaved changes, the GUI first asks *"Save all changes?"* — **Save**, **Discard** or **Cancel** — so a batch never silently runs a stale version.

I.3.4 Running simulations

Run executes every non-ignored file in the directory in turn (a batch queue). Progress for the current file is shown in the progress bar and the status line. Use **Pause**/**Unpause** to suspend and continue, and **Cancel** to abort. **Resume run** continues a previously interrupted run from its latest checkpoint.

I.3.5 The log pane

Log output appears in the read-only pane at the bottom, with colour coding: **ERROR** lines in red, **WARNING** lines in orange, and "Building ..." lines (JIT build / per-file banners) in green so they stand out.

Chapter I.4

Settings: Simulation

The **Simulation Settings** sub-tab (inside the Cockpit's settings tree) controls how the run is integrated and how atoms are injected (Figure I.4.1).

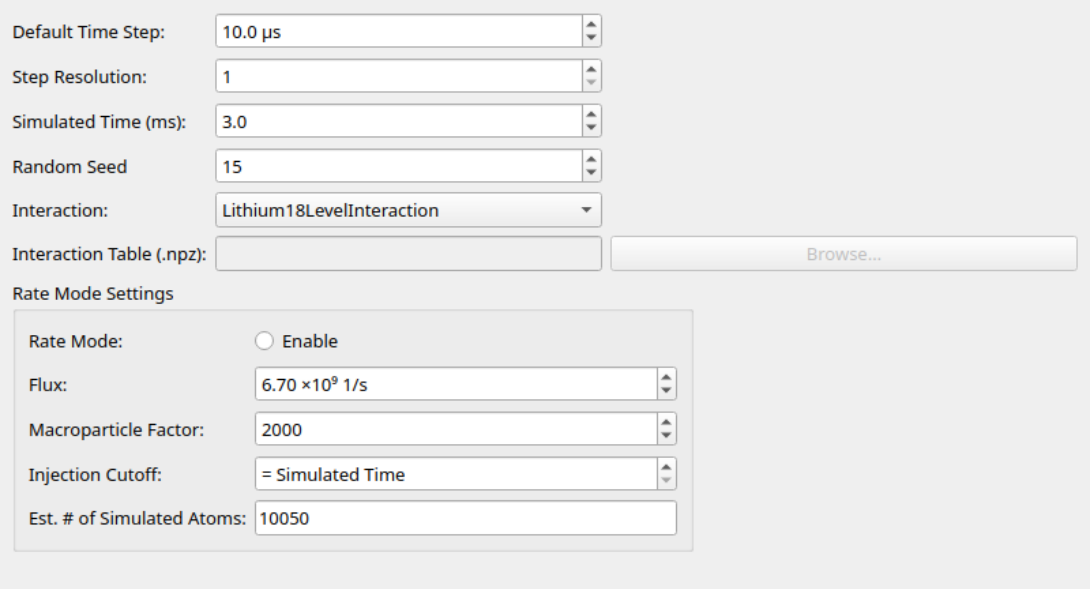


Figure I.4.1: The Simulation Settings sub-tab.

I.4.1 Run parameters

Default Time Step (µs) The outer step over which the engine advances all atoms and writes one row of output per atom.

Step Resolution A legacy parameter that no longer has any effect. See the note below.

Simulated Time (ms) Total simulated duration of the run.

Random Seed Seed for the random number generator; fix it for reproducible runs.

Interaction The light–atom interaction model — how many atomic levels are tracked and which transitions between them are allowed. Simple models (e.g. a 2-level cycling approximation) are fast; the full 18-level D2 hyperfine model is more faithful but slower. The list is populated automatically from the classes in `src/interactions.py` (see the Technical Manual, *Interaction models*).

Interaction Table (.npz) Path to a pre-computed lookup table. This row is enabled *only* when the interaction is set to `Lithium6DiagonalizerTableInteraction`; the table is produced by the Diagonalizer Tables tab (chapter I.13).

Deep dive: the timestep is not the physics resolution

The **Default Time Step** is the *outer* step: at its end every atom advances by that amount and one output row is written per atom. Inside each outer step the engine does not take a single jump. It runs an event-driven inner loop that samples scattering events one at a time and subdivides the motion into as many *substeps* as needed — in particular so the magnetic field an atom feels does not change too much between substeps. So a smaller Default Time Step gives you finer output and time resolution, but the physical accuracy of the trajectory is governed by this internal substepping, not by the outer step alone.

Known issue

Step Resolution is currently not wired up. The value is validated, stored, and copied into the run's `config.json`, but it does not affect the simulation or how often results are written — every step is written. Historically it set the disk-write interval (write every N-th step).

I.4.2 Rate mode

Rate mode switches from a fixed atom sample to *continuous injection* at a given flux. Tick **Rate Mode: Enable** to activate it; the flux, macroparticle and injection-cutoff fields are only editable when it is on.

Flux ($\times 10^9$ 1/s) The atom injection rate.

Macroparticle Factor How many real atoms each simulated (macro)particle represents.

Injection Cutoff (ms) Time at which injection stops. Entering **0** (displayed as *"= Simulated Time"*) injects for the whole run.

Est. # of Simulated Atoms Read-only estimate, computed as round($\Phi\, t/w$), where Φ is the flux, t the simulated time and w the macroparticle factor. It updates automatically and cannot be edited.

Deep dive: why macroparticles?

A real 2D MOT is loaded by a beam with a flux of order 10^9–10^{11} atoms per second. Simulating each atom individually would be intractable. Instead, each simulated *macroparticle* stands in for w real atoms (the **Macroparticle Factor**): the engine injects only Φ/w macroparticles per second, follows those, and any extensive quantity (atom counts, densities) is scaled back up by w afterwards. Larger w means fewer, cheaper macroparticles but coarser statistics; smaller w is more faithful but slower. This is why the estimated atom count is round($\Phi\, t/w$): it counts macroparticles, not real atoms.

Note

The estimated atom count is derived from the fields above; change flux, simulated time or the macroparticle factor to influence it.

Chapter I.5

Settings: Atoms

The **Atoms** sub-tab defines the atomic species and the initial ensemble of atoms (Figure I.5.1).

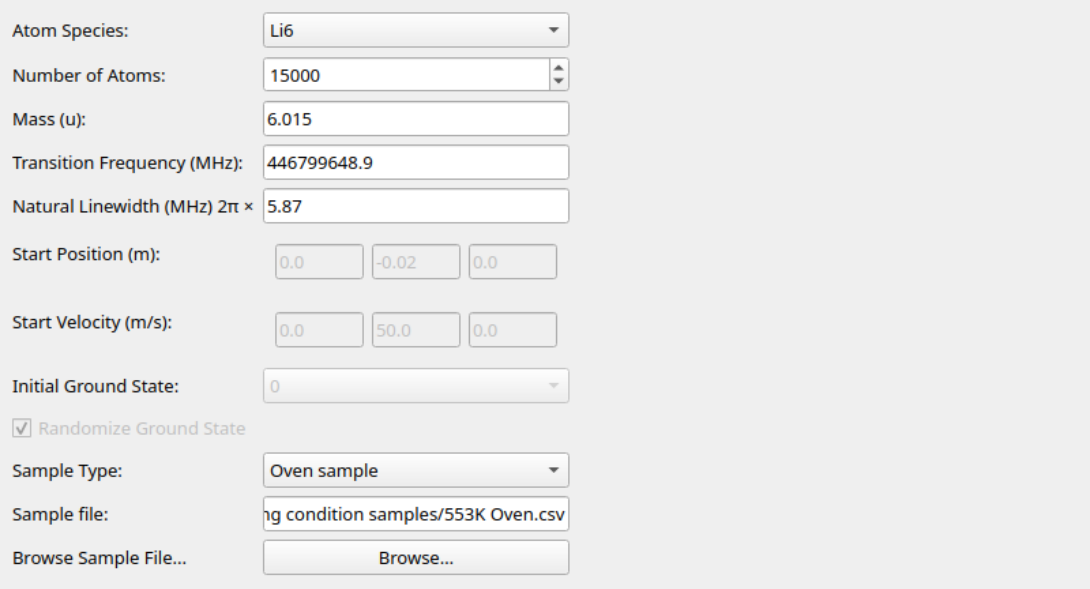


Figure I.5.1: The Atoms sub-tab.

I.5.1 Species and physical constants

Atom Species Chosen from the species defined in `src/atoms.py` (currently `Li6`).

Physical constants *Mass* (u), *Transition Frequency* (MHz) and *Natural Linewidth* (MHz, $2\pi\times$) are read-only: taken from the selected species and shown for reference, they cannot be edited here.

I.5.2 The initial ensemble

Number of Atoms Size of the simulated ensemble.

Start Position (m) Initial position $[x, y, z]$ common to all atoms.

Start Velocity (m/s) Initial velocity $[x, y, z]$.

Initial Ground State Index (0–5) into the interaction model's list of ground states.

Randomize Ground State When ticked, each atom is assigned a random ground state instead of the fixed one; this disables the *Initial Ground State* selector.

Deep dive: what the ground state means

The interaction model splits the atomic ground level into several sublevels (the hyperfine / Zeeman states) — six of them for the Li-6 D2 models. An atom sits in one of these at any time, and which one it is affects which transitions it can be driven on. Fixing the **Initial Ground State** starts every atom in the same sublevel; **Randomize Ground State** spreads them across all sublevels, closer to an unpolarized thermal population. During the run atoms move between sublevels as they scatter photons.

I.5.3 Loading a sample

Instead of a single common initial condition, you can load a sample that provides per-atom positions and velocities.
Sample Type *Oven sample* or *Snapshot sample*.
Sample file Path to a CSV sample (use **Browse...**).
Oven samples are produced by the Sample Generator (chapter I.9); snapshot samples capture the state of a previous run.

Deep dive: how atoms enter the simulation

A sample supplies a position, velocity and internal state for every atom, so the ensemble is no longer a single common starting point but a realistic spread of initial conditions. In ordinary runs all atoms are present from the start. In *rate mode* (chapter I.4) they are instead "born" gradually over the run: each atom carries a scheduled entry time and only becomes active once the simulation clock reaches it, reproducing a steady loading flux rather than a single pulse.

Note

When a **Sample file** is set, the *Start Position*, *Start Velocity*, *Initial Ground State* and *Randomize Ground State* fields are disabled — the sample supplies those per-atom initial conditions.

Chapter I.6

Settings: Lasers

The **Lasers** sub-tab edits the list of laser beams. Each row of the table is one beam; the panel on the left adds, removes and bulk-edits beams (Figure I.6.1).

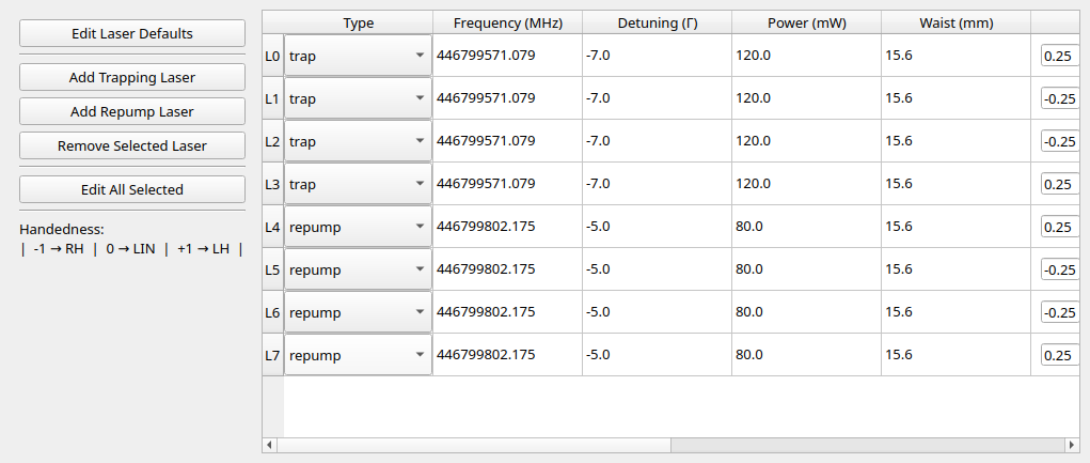


Figure I.6.1: The Lasers sub-tab: control panel (left) and the per-beam table (right).

I.6.1 The beam table

Column	Meaning
Type	`unspecified`, `repump` or `trap`
Frequency (MHz)	Absolute beam frequency
Detuning (Γ)	Detuning in units of the natural linewidth
Power (mW)	Beam power
Waist (mm)	Beam waist
Origin	Beam origin point $[x, y, z]$ (m)
Direction	Propagation direction $[x, y, z]$
Handedness	Polarization: -1 RH, 0 LIN, $+1$ LH
t_on (ms)	Time the beam switches on
t_off (ms)	Time it switches off; leave empty to stay on all run

Deep dive: how a beam acts on an atom

The columns above are not independent labels — together they determine how strongly each atom scatters photons from the beam.

- **Power** and **Waist** set the intensity. The beam has a Gaussian profile, so an atom off the beam axis sees less intensity than one on it. Higher intensity drives faster scattering, up to a saturation limit.
- **Frequency/Detuning** set the tuning *as the atom sees it.* The relevant quantity is the detuning after the atom's own Doppler shift (from its velocity) and Zeeman shift (from the local field) are added in — this is what makes the force velocity- and position-dependent, and hence trapping.
- **Origin/Direction** fix where the beam is and which way its photons push; **Handedness** sets the polarization, which selects which transitions the beam can drive.

I.6.2 Managing beams

Add Trapping Laser / Add Repump Laser Append a new beam, pre-filled from the defaults in `GUI/defaults/lasers/` (`trap_default.json` or `repump_default.json`).
Remove Selected Laser Delete the highlighted row.
Edit All Selected Bulk-edit a property across beams via a popup.
Edit Laser Defaults Edit the default templates used when adding beams.

Note

Handedness sets the beam polarization: $+1$ is left-handed (helicity $+1$, i.e. σ^+ for $\mathbf{k} \parallel \mathbf{B}$), -1 is right-handed, and 0 is linear. Leaving **t_off** empty means the beam never turns off during the run.

Chapter I.7

Settings: Magnetic Field

The **Magnetic Field** sub-tab selects the field model and edits its parameters. Choosing a different *Field Type Selection* rebuilds the parameter fields below to match the chosen model (Figure I.7.1).

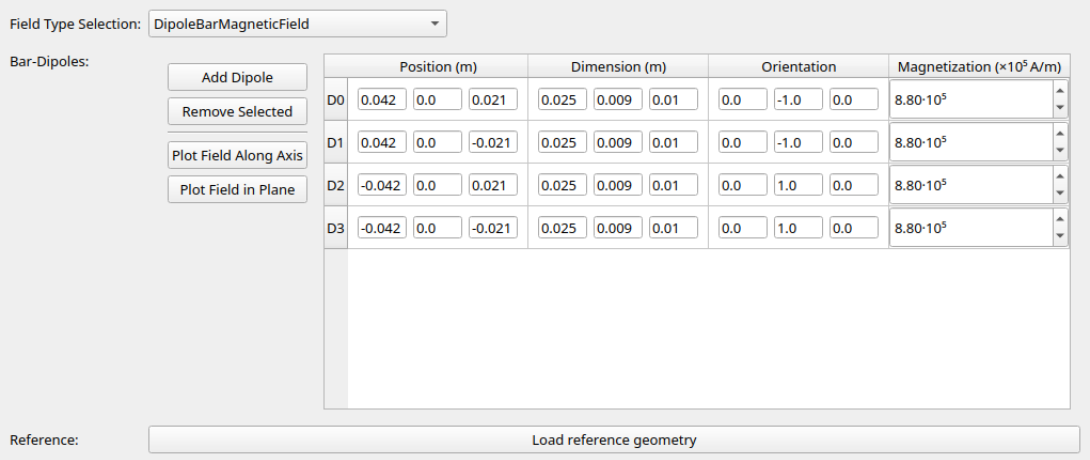


Figure I.7.1: The Magnetic Field sub-tab.

I.7.1 Choosing a model

The **Field Type Selection** dropdown lists *No Magnetic Field* plus every field class defined in `src/magnetic_field.py`. Only the selected model's parameters are shown. The physics of each model, and how to add a new one, are covered in the Technical Manual (*Magnetic-field models* and *Adding a new magnetic-field model*).

Deep dive: why the field shape matters

The magnetic field is what makes a MOT trap rather than just cool. Through the Zeeman shift it makes an atom's transition frequency depend on *where the atom is*: the further an atom drifts from the field zero, the more one circular polarization is pushed into resonance and the other out of it, so the net photon force points back toward the centre. The field model and its gradient therefore set the size, position and stiffness of the trap — which is why the geometry here is worth getting right.

I.7.2 Parameters by model

No Magnetic Field No parameters.

IdealQuadrupoleField *Gradient* (T/m) and *Center offset* (mm, $[x, y, z]$).

EllipticalMagneticField Gradients *g_x*, *g_y* (T/m), *Tilt angle* θ (degrees) and *Center offset* (mm).

ZeemanField *Length of the Slower* (m), *Maximum Magnetic Field* B_0 (T), *Bias Field* B_{bias} (T), *Max. Change in Magnetic Field* (%) and *Min. Change in Magnetic Field* (T).

Bar-dipole models `DipoleBarMagneticField` and `CuboidBarMagneticField`: a table of bar dipoles defining the magnet geometry, plus a **Load reference geometry** button.

GridFieldModel A *Grid file* (.npz) holding a pre-computed field grid, a fallback bar-dipole table, a *Center offset*, and a **Load reference geometry** button.

Note

Angles are entered in degrees (converted to radians internally). **Load reference geometry** fills the bar-dipole table from a stored arrangement in `GUI/defaults/magnets/`.

Chapter I.8

Settings: Boundaries

The **Boundaries** sub-tab defines the simulation volume. An atom that leaves the box is removed from the simulation (Figure I.8.1).

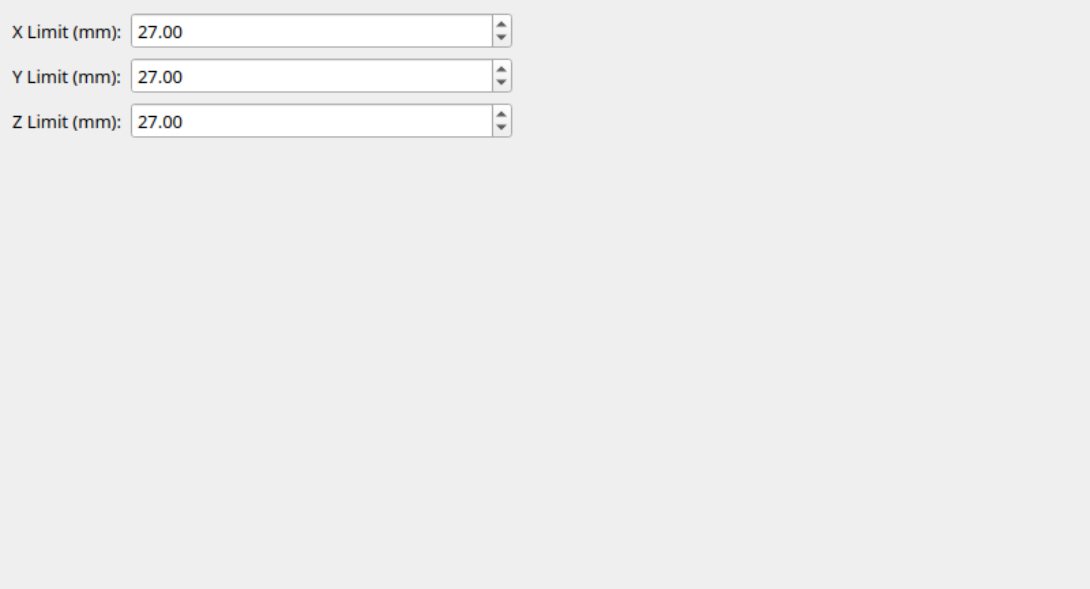


Figure I.8.1: The Boundaries sub-tab.

I.8.1 Box limits

The three fields — **X Limit**, **Y Limit** and **Z Limit** (all in mm) — set the extent of the simulation box along each axis. Each limit is a symmetric half-extent about the origin: an atom is removed as soon as $|x| \geq x_{\text{limit}}$, $|y| \geq y_{\text{limit}}$ or $|z| \geq z_{\text{limit}}$. A limit therefore describes the distance from the centre to the wall, not the full box width.

Note

The Boundaries tab is currently limited to the three symmetric box limits above. More operational boundary handling (per-face and geometry-aware kills) is planned for version 1.2.0.

Known issue

There is a known issue in the boundary handling for the `ZeemanField` model (a `FIXME: Zeeman Boundaries` on the y-boundary kill in `src/simulate.py`). See the Technical Manual for details.

Chapter I.9

The Sample Generator

The **Sample Generator** tab produces the CSV sample files that the Atoms sub-tab can load (chapter I.5). It has two modes, selected by the radio buttons at the top left: *Create Sample from Oven* and *Create Sample from File* (Figure I.9.1).

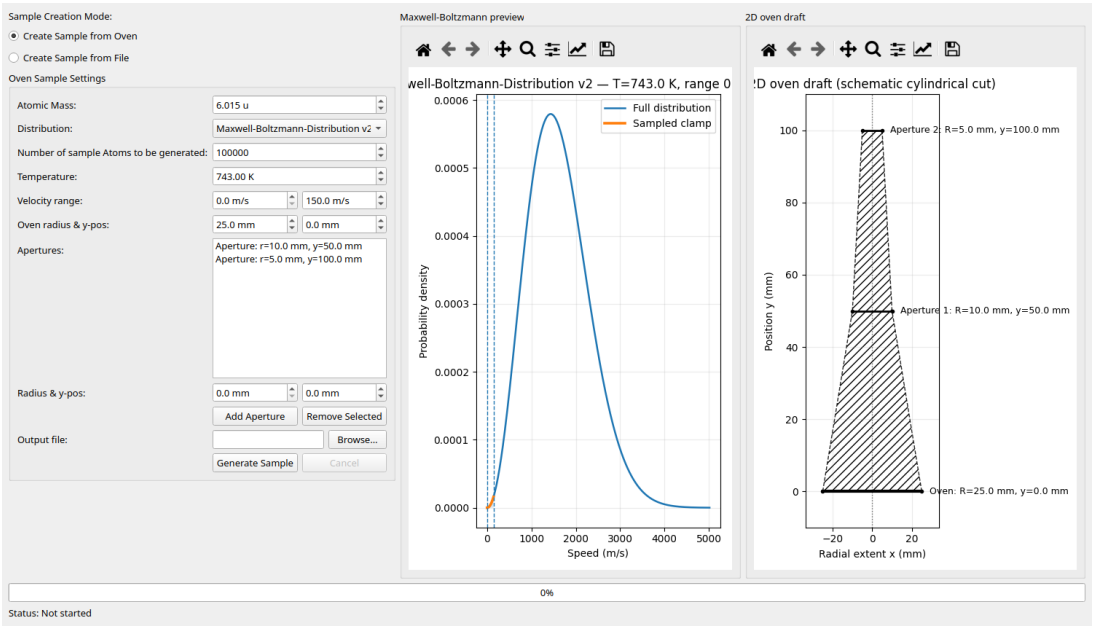


Figure I.9.1: The Sample Generator in oven mode, with the Maxwell–Boltzmann preview and 2D oven draft on the right.

I.9.1 Oven mode

Generates a Maxwell–Boltzmann velocity sample emitted from an oven through a set of apertures.

Atomic Mass (u) Mass of the sampled species.

Distribution Which Maxwell–Boltzmann speed distribution to sample from: *v2* ($\propto v^2$) or *v3* ($\propto v^3$). See the deep dive below.

Number of sample Atoms How many atoms to generate.

Temperature (K) Oven temperature.

Velocity range (m/s) Minimum and maximum speed retained in the sample.

Oven radius & y-pos (mm) Geometry of the oven source.

Apertures A list of apertures (each with a radius and y-position); use **Add Aperture** / **Remove Selected** to edit it.
Output file Destination CSV (via **Browse…**).
Two live previews update as you edit: the **Maxwell–Boltzmann preview** (with the selected velocity range highlighted) and the **2D oven draft** (a schematic cut through the oven and apertures). **Generate Sample** runs the generation in the background — progress is shown in the bar at the bottom and it can be stopped with **Cancel**.

Deep dive: v2 vs. v3, and what the apertures do

The two distributions describe different physical situations. *v2* ($\propto v^2$) is the ordinary speed distribution of atoms *inside* a gas at temperature T. *v3* ($\propto v^3$) is the *effusive* (flux-weighted) distribution of atoms *leaving* an oven aperture: faster atoms cross the opening more often, so the emitted beam is weighted by an extra factor of v. For a sample meant to represent an atomic beam from an oven, *v3* is the physically correct choice.
The **apertures** model the collimation of that beam. Only atoms whose trajectory passes through every aperture opening are kept, so the aperture radii and y-positions set the angular spread (solid angle) of the beam that reaches the trap — narrower or more distant apertures produce a more collimated, lower-divergence sample.

I.9.2 File mode

Extracts a single time-step snapshot from a previous simulation's `result.csv` and writes it as a sample.
Input file A result CSV; it must contain the per-atom columns `atom_id`, `subjective_time`, `position_x/y/z`, `velocity_x/y/z`, `excitation_count` and `current_groundstate`.
Step The step index to extract; the corresponding time (ms) is shown next to it.
Output file Destination CSV.
The preview plots the average position and velocity by step, with the selected step marked. **Generate Sample** writes the per-atom state at that step.

Note

Oven samples are loaded as *Oven sample* in the Atoms tab; file snapshots are loaded as *Snapshot sample*.

Chapter I.10

The Incrementor

The **Incrementor** generates many JSON configuration files by sweeping one or more parameters across template files. The result is the Cartesian product of the templates and every queued sweep — convenient for parameter scans that you then run as a batch from the Cockpit (Figure I.10.1).

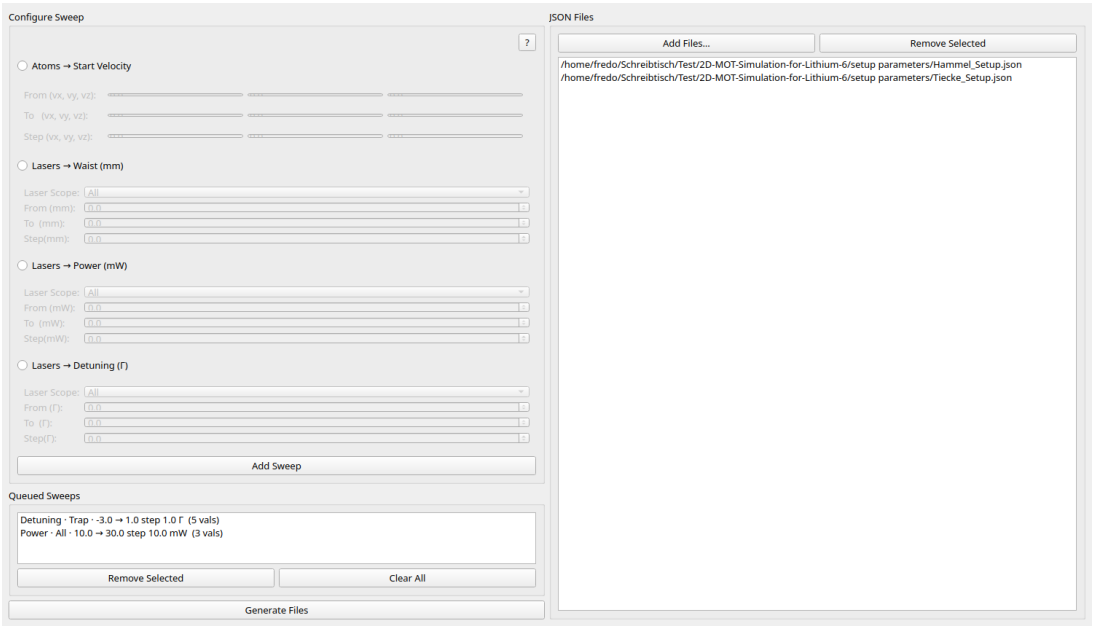


Figure I.10.1: The Incrementor with two templates loaded (right) and two sweeps queued (left): a trap-beam detuning sweep ($-3 \to +1\,\Gamma$, 5 values) and an all-beam power sweep ($10 \to 30\,\mathrm{mW}$, 3 values).

I.10.1 Workflow

1. On the right, **Add Files...** — the JSON templates to vary.
2. On the left, choose one sweep type and fill in its *From*, *To* and *Step*, then click **Add Sweep**. It appears in **Queued Sweeps**.
3. Repeat to combine several axes.
4. Click **Generate Files**, pick a target directory, and confirm.

I.10.2 Sweep types

Atoms → Start Velocity Independent *From/To/Step* vectors for v_x, v_y, v_z. Each axis with *Step* > 0 becomes its own queued entry.
Lasers → Waist (mm) Swept over a *Laser Scope* (All / Trap only / Repump only).
Lasers → Power (mW) As above.
Lasers → Detuning (Γ) As above.

I.10.3 Generation

The number of files written is

$$\text{len(files)} \times \prod_{\text{sweeps}} \text{(values per sweep)},$$

and the tool asks for confirmation before writing. Each output filename is tagged with the swept values. Conflicting sweeps are rejected: the same velocity axis twice, or two laser sweeps of the same parameter whose scopes overlap (*All* overlaps everything).

Note

Worked example (Figure I.10.1). Two templates, a 5-value detuning sweep and a 3-value power sweep give $2 \times 5 \times 3 = 30$ output files. Each filename is the template name with the swept values appended, e.g. `Hammel_Setup` with detuning $-3\,\Gamma$ (trap) and power 10 mW becomes `Hammel_Setup_trap_-3.0Gamma_beam_power_10.0mW.json`.

Note

The **?** button in the sweep panel shows an in-app summary of this workflow.

Chapter I.11

The Plotting Tab

Not implemented

The **Plotting** tab is not implemented. It currently displays only a placeholder label ("Plotting Tab – Shell", Figure I.11.1). Analysis and plotting are instead done with the standalone scripts in the `util/` directory.

Figure I.11.1: The Plotting tab is an empty placeholder shell.

Chapter I.12

The Spectrum Tab

Work in progress

The Spectrum tab is still under active development and not yet final; its controls and outputs may still change.

The **Spectrum** tab computes a simulated absorption spectrum: it takes a snapshot of a simulation, defines a spectroscopy probe beam, and scans the beam frequency to produce a spectrum (Figure I.12.1). The input panel is on the left, the resulting plots on the right.

Deep dive: what the spectrum represents

The spectrum is a stationary probe of one frozen instant of a run. The atoms' positions and velocities are held fixed at the chosen snapshot; only the probe beam's frequency is varied. At each frequency the tool sums how strongly the ensemble would scatter, and plotting that sum against frequency traces out an absorption line shape. Its width and position encode the velocity spread and any Zeeman shifts present in the snapshot — the same information an experimentalist reads from an absorption-spectroscopy scan.

I.12.1 Simulation snapshot

Input CSV A simulation `result.csv` (same required columns as the Sample Generator's file mode, chapter I.9).
Step The time-step to analyse; its time (ms) is shown alongside.
Bin count Number of bins for the velocity histograms (distinct from the scan *Bin count* in the Scan range section below).
Exclude states Comma-separated ground-state indices to omit from the snapshot (e.g. `0,2`).

I.12.2 Spectroscopy beam

Gaussian beam profile The checkbox *"Apply Gaussian beam profile at atom positions"*, when ticked, weights atoms by a Gaussian beam intensity at their positions; this enables the beam *Origin*.
Origin (m), Direction Beam geometry.
Power (mW), Frequency (MHz), Detuning (Γ) Beam strength and tuning.
Handedness Probe polarization (−1, 0, +1).
Radius (waist) (mm) Beam waist.
Interaction model The interaction model used for the probe.

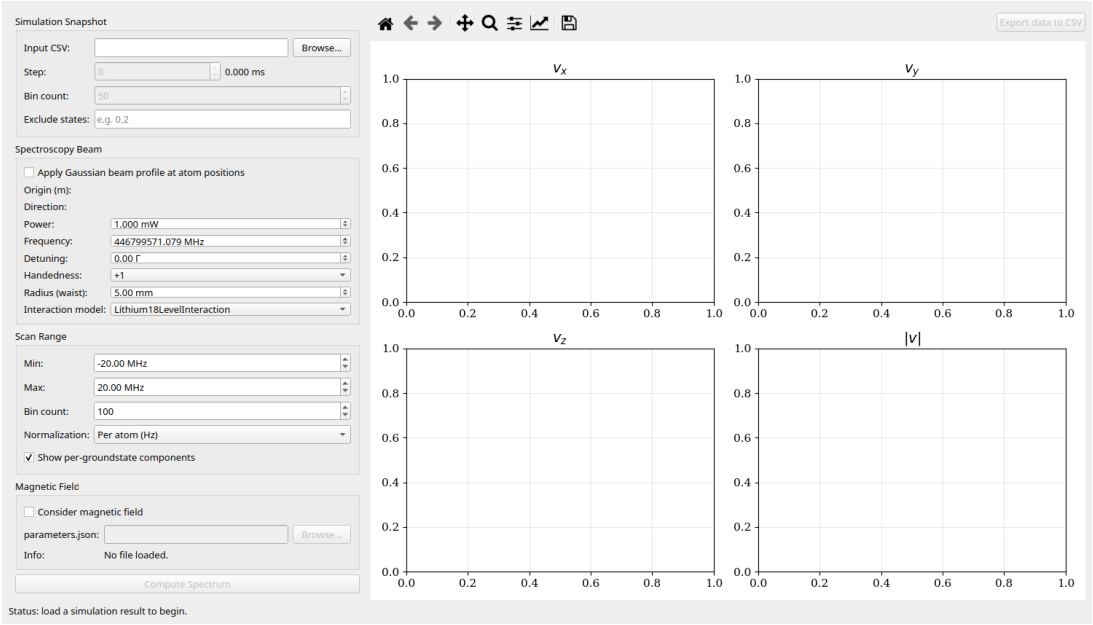


Figure I.12.1: The Spectrum tab: snapshot, beam, scan and field controls (left) and the velocity/spectrum plots (right).

I.12.3 Scan range

Min / Max (MHz) Frequency range of the scan.
Bin count Number of frequency points across the scan (distinct from the snapshot *Bin count* in the Simulation snapshot section).
Normalization How the spectrum is normalized (e.g. per atom, Hz).
Show per-groundstate components Overlay each ground state's contribution.

I.12.4 Magnetic field

Tick **Consider magnetic field** and load a `parameters.json` to include Zeeman shifts from that configuration's field in the scan.

I.12.5 Computing and exporting

Compute Spectrum runs the scan and fills the 2×2 plot grid (velocity distributions and the resulting spectrum). The matplotlib toolbar allows panning/zooming, and **Export data to CSV** saves the computed data.

Chapter I.13

The Diagonalizer Tables Tab

The **Diagonalizer Tables** tab generates an interaction lookup table (an `.npz` file) by diagonalizing the atomic Hamiltonian over a range of magnetic-field magnitudes. The table is then consumed at run time by the `Lithium6DiagonalizerTableInteraction` model, selected in the Simulation settings (chapter I.4). The settings are on the left, a preview plot on the right (Figure I.13.1).

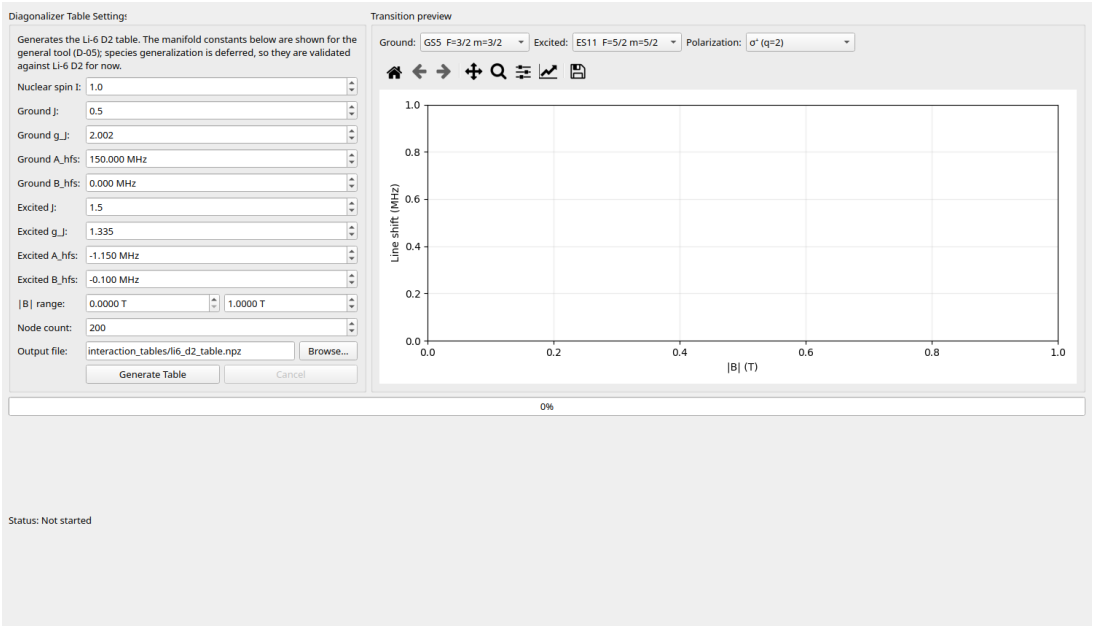


Figure I.13.1: The Diagonalizer Tables tab.

Known issue

This tool currently generates only the **Li-6 D2** table. The manifold constants below are exposed for the eventual general tool, but species/line generalization is deferred — the worker validates the entered fields against Li-6 D2.

Deep dive: why a table?

At zero field an atom's states are the familiar hyperfine levels; at very strong field they are decoupled electron- and nuclear-spin states. In between — exactly the regime a MOT operates in — neither picture holds: the sublevels *mix*, and both their energies and the strengths of the optical transitions between them change continuously with the field magnitude $|B|$.

Getting the scattering right therefore means diagonalizing the atomic Hamiltonian at the local $|B|$. Doing that for every atom at every substep would be far too slow, so this tab does it once on a grid of $|B|$ values and stores the result. At run time the interaction model simply looks up (and interpolates) the pre-computed eigenstates and transition strengths.

I.13.1 Settings

Nuclear spin I Shared nuclear spin.
Ground manifold J, g_J, A_{hfs} (MHz), B_{hfs} (MHz).
Excited manifold J, g_J, A_{hfs} (MHz), B_{hfs} (MHz).
$|B|$ **range (T)** Minimum and maximum field magnitude spanned by the table.
Node count Number of grid points across the $|B|$ range.
Output file Destination `.npz` (default `interaction_tables/li6_d2_table.npz`).
Generate Table runs the diagonalization in the background (progress bar and status line at the bottom); **Cancel** stops it.

I.13.2 Using the table

Set the Simulation interaction to `Lithium6DiagonalizerTableInteraction` and point *Interaction Table (.npz)* at the generated file (chapter I.4).

Chapter I.14

Running from the Command Line

The same entry point, `python -m main`, launches either the GUI or a headless batch run.

I.14.1 GUI mode

```
python -m main --GUI
python -m main --GUI --style dark
```

I.14.2 Batch mode

Pass one or more parameter files with `--files`; results go under `--target-dir`:

```
python -m main --files "setup parameters/Hammel_Setup.json" --target-dir my_results
```

Progress is printed to the console. File paths are resolved relative to the current working directory.

I.14.3 Options

Option	Values	Purpose
`-s`, `--style`	`light` (default), `dark`	GUI theme
`--GUI`	flag	Launch the GUI
`--files`	one or more paths	Parameter files to run in batch
`--target-dir`	path	Output directory (default `2D-MOT-Simulation-For-Lithium-6`)

Note

Running without `--GUI` and without `--files` prints a usage hint and exits. As in the GUI, the first batch run triggers Numba JIT compilation (1–3 minutes).

Chapter I.15

Output Files

Each run writes a self-contained folder:

```
simulation_results/DD_MM_YY_N/run_N/
  result.csv # per-step, per-atom state
  config.json # copy of the parameters used
```

I.15.1 result.csv

One row per alive atom per written step. Every run writes all of the columns below (per-column output flags exist internally but are not user-configurable):

Column	Meaning
`step`	Step index
`atom_id`	Atom identifier
`position_x/y/z`	Position (m)
`velocity_x/y/z`	Velocity (m/s)
`subjective_time`	Per-atom elapsed time (s)
`excitation_count`	Number of excitations so far
`current_groundstate`	Current ground-state index

These are exactly the columns that the Sample Generator's file mode (chapter I.9) and the Spectrum tab (chapter I.12) expect as input.

I.15.2 config.json

A verbatim copy of the parameter file used for the run, so every result folder is reproducible and self-describing.

Chapter I.16

Troubleshooting

Common first-run questions and what to do about them.

I.16.1 The first run hangs for minutes before anything happens

The physics core is compiled with Numba the first time it runs, which takes 1–3 minutes. This happens once: subsequent runs use the on-disk cache and start immediately. The cache is rebuilt automatically whenever the compiled code changes, so an occasional slow run after an update is expected.

I.16.2 A file shows "N error(s)" in the Status column

The parameters failed schema validation. Open the validation/diff view from the file table (chapter I.3) to see which fields are invalid, fix them in the settings tree, and the count updates. A batch skips nothing on its own account, so resolve the errors before running or tick *Ignore* on the offending file.

I.16.3 The command line prints a usage hint and exits

Running `python -m main` without `--GUI` and without `--files` has nothing to do, so it prints a hint and exits. Pass `--GUI` to launch the interface or `--files` for a headless batch run (chapter I.14).

I.16.4 I can't find my results

Each run writes to `simulation_results/DD_MM_YY_N/run_N/` (chapter I.15). From the command line the parent output directory is set by `--target-dir` and resolved relative to the current working directory, not the repository root.

Chapter I.17

Known Issues and Limitations

This chapter collects the current limitations of the software in one place. Each entry links back to the chapter that covers it in context. Colour-coded callouts flag the same items where they appear in the text.

Area	Status	Detail
Plotting tab	Not implemented	Empty placeholder shell; use the analysis scripts in `util/` instead (chapter I.11).
Diagonalizer Tables	Li-6 D2 only	Species/line generalization is deferred; the worker validates entered manifold constants against Li-6 D2 (chapter I.13).
Step Resolution	Dead parameter	Validated, stored and copied into `config.json` but never read — every step is written (chapter I.4).
Boundaries (`ZeemanField`)	Known bug	A `FIXME` on the y-boundary kill for the Zeeman-slower field (chapter I.8).
Atom–atom interactions	Not modelled	Collisions are not yet included in the physics (Introduction).
Spectrum tab	Under development	Present and usable, but controls and outputs may still change (chapter I.12).

Note

Planned features. Expanded boundary handling is scheduled for version 1.2.0 (chapter I.8), and atom–atom interactions (collisions) are planned as a future addition to the physics.